

CASE TÉCNICO - DESENVOLVIMENTO DE SISTEMA EMBARCADO

Sistema de Monitoramento de Recursos e Periféricos Embarcados

1. Objetivo

Desenvolver uma aplicação para ambiente Linux capaz de monitorar recursos do equipamento embarcado e periféricos conectados. O sistema deverá possuir interface Web, back-end desenvolvido em Java e persistência utilizando banco de dados.

2. Ambiente de Desenvolvimento

A aplicação deverá ser desenvolvida obrigatoriamente para execução em ambiente Linux.

Distribuições sugeridas:

- Ubuntu 22.04 LTS (preferencialmente)
- Debian 12

O desenvolvimento poderá ser realizado utilizando:

- NetBeans
 - IntelliJ IDEA
-

3. Tecnologias Obrigatórias

- Back-end
 - Desenvolver utilizando:
 - Java 8 ou Java 17
 - Spring Boot
 - Spring Security
 - Spring Data JPA / Hibernate
 - Maven ou Gradle
 - Front-end
 - Desenvolver utilizando:
 - Vue.js
 - Axios
 - Vue Router
 - Banco de Dados
 - PostgreSQL
-

4. Estrutura Geral do Sistema

O sistema deverá possuir os seguintes módulos:

- Login (Não obrigatório)
 - Dashboard (Não obrigatório)
 - Configurações gerais (Não obrigatório)
 - Cadastro de Câmeras (Não obrigatório)
 - Monitoramento de Internet
 - Monitoramento de Disco
 - Monitoramento das Câmeras
 - Visualização do Stream
-

5. Autenticação (Não obrigatório)

O sistema deverá possuir uma tela de Login contendo:

- Usuário
- Senha

Após autenticação, o usuário deverá acessar o Dashboard.

As senhas deverão ser armazenadas utilizando criptografia segura (BCrypt ou equivalente).

6. Dashboard (Não obrigatório)

Após realizar login, o usuário deverá visualizar um Dashboard contendo, no mínimo:

- Status da Internet
- Status do Disco
- Espaço utilizado
- Espaço livre
- Percentual de utilização
- Quantidade de câmeras cadastradas
- Quantidade de câmeras online
- Quantidade de câmeras offline
- Última execução das rotinas
- Alertas recentes

É desejável a utilização de gráficos.

7. Configurações Gerais (Não obrigatório)

O sistema deverá possuir uma tela para configurações gerais contemplando:

- Configuração para monitoramento de internet
 - IP para teste de internet
 - Timeout
 - Periodicidade
- Configuração para monitoramento de disco.
 - Limiares para geração de alertas
 - Periodicidade
- Configuração para monitoramento das câmeras
 - Periodicidade

As configurações devem ser persistidas em banco.

8. Monitoramento da Internet

A aplicação deverá executar automaticamente uma rotina periódica para verificar a disponibilidade da Internet.

Cada verificação deverá armazenar:

- Data/Hora
 - Status
 - Tempo de resposta
 - Mensagem de erro (quando existir)
-

9. Monitoramento do Disco

A aplicação deverá monitorar periodicamente o armazenamento do equipamento.

Registrar:

- Espaço total
- Espaço utilizado
- Espaço livre
- Percentual utilizado
- Data/Hora

Caso ultrapasse um limite configurável deverá ser gerado um alerta.

10. Cadastro de Câmeras (Não obrigatório)

O sistema deverá possuir uma tela para cadastro de câmeras IP.

Campos obrigatórios:

- Nome
- Descrição
- Endereço IP
- Porta HTTP

- Porta RTSP
 - Usuário
 - Senha
 - Status (Ativa/Inativa)
-

11. Monitoramento das Câmeras

Para todas as câmeras ativas deverá existir uma rotina periódica responsável por verificar sua disponibilidade.

A rotina deverá verificar:

- Ping no IP da câmera
- Verificar canal RTSP obtendo ao menos um frame

A câmera somente será considerada disponível quando ambas as verificações forem bem sucedidas.

Cada execução deverá ser armazenada no banco de dados:

- Data/hora
- Status
- Tempo do ping
- Resultado da captura do frame
- Mensagem de erro (quando existir)

Caso a câmera fique indisponível deverá ser registrado um alerta.

Quando voltar ao funcionamento normal deverá ser registrado um evento de recuperação.

12. Visualização do Stream

O sistema deverá possuir uma tela para visualização do vídeo das câmeras cadastradas.

O usuário deverá:

- Selecionar uma câmera
- Visualizar seu vídeo
- Visualizar informações da câmera
- Receber indicação caso a câmera esteja offline

Dica:

Como navegadores não reproduzem RTSP diretamente, recomenda-se a utilização do OpenCV no Back-end Java para captura do stream RTSP.

O candidato poderá utilizar outra solução técnica (FFmpeg, HLS, WebRTC, etc.), desde que justifique tecnicamente sua escolha.

13. Banco de Dados

Modelagem mínima esperada:

- CONFIGURACAO
- CAMERA
- MONITORAMENTO_CAMERA
- MONITORAMENTO_DISCO
- MONITORAMENTO_INTERNET
- ALERTA

O candidato poderá acrescentar outras tabelas caso julgue necessário.

14. API REST

A aplicação deverá disponibilizar APIs REST para todas as funcionalidades do sistema.

Será obrigatório a disponibilização de rotas que entreguem as informações de monitoramentos (status e métricas) e alertas - Dados que são exibidos na tela de Dashboard. Espera-se utilização adequada dos métodos HTTP:

- GET
- POST
- PUT
- DELETE

Será obrigatório disponibilizar as informações de rotas e estruturas da API através de Swagger/OpenAPI

15. Requisitos Não Funcionais

A aplicação deverá apresentar:

- Arquitetura organizada
 - Separação em camadas
 - Código limpo
 - Tratamento de exceções
 - Boas práticas de orientação a objetos
 - Boas práticas REST
 - Segurança
 - Facilidade de manutenção
-

16. Qualidade de Código

Será considerado na avaliação:

- Organização do projeto
- Nomenclatura
- Clareza do código
- Reutilização
- Princípios SOLID
- Padrões de Projeto quando aplicáveis

O código deverá possuir comentários explicativos sempre que necessário, principalmente em trechos de maior complexidade.

As principais classes e métodos públicos deverão possuir documentação utilizando **Javadoc**.

17. Testes

Espera-se a implementação de testes unitários para as principais regras de negócio.

Os testes deverão contemplar, sempre que possível:

- Monitoramento da Internet
- Monitoramento do Disco
- Monitoramento das Câmeras
- Regras de geração de alertas

A utilização de JUnit 5 e Mockito é recomendada, bem como testes de integração.

18. Documentação

O projeto deverá conter um README contendo:

- Descrição da solução
- Arquitetura utilizada
- Requisitos para execução
- Dependências
- Como compilar
- Como executar
- Usuário padrão
- Estrutura do banco
- Tecnologias utilizadas
- Justificativa para as principais decisões técnicas

19. Diferenciais (Não Obrigatórios)

Serão considerados diferenciais:

- Docker Compose
 - Flyway ou Liquibase
 - JWT
 - Refresh Token
 - Gráficos no Dashboard
 - Atualização em tempo real utilizando WebSocket
 - Health Check da aplicação
 - Cache
 - Criptografia das senhas das câmeras
 - Internacionalização
 - Responsividade da interface
 - Testes de Integração
 - Pipeline CI/CD
 - Monitoramento utilizando Micrometer/Prometheus
-

20. Entregáveis

O candidato deverá entregar:

- Código-fonte completo;
 - Projeto do Back-end;
 - Projeto do Front-end;
 - Scripts SQL ou migrations para criação do banco;
 - README com instruções completas de instalação e execução;
 - Evidências de funcionamento (prints ou vídeo);
 - Testes implementados.
 - Swagger/OpenAPI
-

21. Prazo

Prazo sugerido para desenvolvimento: **3 a 5 dias corridos.**

22. Observações

Durante a avaliação, serão analisados não apenas o funcionamento da aplicação, mas principalmente a qualidade técnica da solução apresentada.

Mais importante do que implementar todas as funcionalidades é demonstrar capacidade de projetar um software organizado, escalável, seguro e de fácil manutenção, utilizando boas práticas de desenvolvimento.

Caso alguma decisão técnica seja diferente das sugestões apresentadas neste documento, ela poderá ser adotada desde que esteja devidamente justificada na documentação entregue.